

Akıllı Depo Yönetimi — Modül Geliştirme Testi

Genel Bilgi

Bu doküman, **Akıllı Depo Yönetimi** modülünün sıfırdan geliştirilmesini kapsamaktadır. Amaç; belirlenen teknoloji yığını ve mimari kurallar çerçevesinde çalışan bir backend API ve frontend arayüzü oluşturmaktır.

Yapay zeka araçları kullanılabilir, ancak kullanılan tüm teknoloji, framework ve yöntemlerin bilinmesi **zorunludur**.

ÖNEMLİ: Bu görevde **özgün tasarım ve hayal gücü** beklenmektedir. Veritabanı yapısı, entity tasarımı, iş akışı mantığı ve kullanıcı arayüzü tamamen size bırakılmıştır. Aşağıda yalnızca temel senaryo ve uyulması zorunlu teknik kurallar belirtilmiştir. Sistemin nasıl modelleneceği, hangi alanların olacağı, depo içeriğinin ne olacağı ve kullanıcı deneyiminin nasıl şekilleneceği sizin kararınızdır. Birbirinin kopyası projeler değerlendirme dışı bırakılacaktır.

Tahmini süre: ~1 iş günü (8 saat)

Teknoloji Gereksinimleri

Katman	Teknoloji
Backend	.NET 9.0 (ASP.NET Core Web API)
Frontend	React 18 + TypeScript + Material-UI (MUI)
Veritabanı	MS SQL Server
ORM	Entity Framework Core

Mimari Yapı

Proje aşağıdaki katmanlı mimariyi takip etmelidir:

Controller → Manager → Repository → Entity (DbContext)

- Entity:** Veritabanı tablosunu temsil eden model sınıfı
- Repository:** Veritabanı işlemlerini gerçekleştiren katman (EF Core ile)
- Manager:** İş mantığı ve validasyonların yer aldığı katman
- Controller:** HTTP endpoint'lerinin tanımlandığı katman

Görev Tanımı

Bir **akıllı depo yönetim sistemi** geliştirilecektir. Bu sistemin temel senaryosu şu şekildedir:

1. Sisteme bir **ürün** tanımlanabilmeli
2. Tanımlanan ürün **depoya giriş** yapılabilmesi (depoya konulmalı)
3. Depodaki ürün **depodan çıkış** yapılabilmesi (depodan alınmalı)

Bu üç temel adımın ötesinde; deponun nasıl modelleneceği, hangi entity'lerin oluşturulacağı, alanların neler olacağı, depo içeriğinin ne olacağı (raf mı, bölge mi, kategori mi vs.), ürün giriş-çıkış akışının nasıl çalışacağı ve kullanıcı arayüzünün nasıl tasarlanacağı **tamamen size bırakılmıştır**.

Zorunlu Teknik Gereksinimler

Tasarımınız ne olursa olsun, aşağıdaki teknik gereksinimler **mutlaka** karşılanmalıdır:

- Her entity'de **CompanyId** alanı bulunmalı (multi-tenant)
- Silme işlemleri **soft delete** (IsDeleted) ile yapılmalı
- En az bir listeleme endpoint'i **server-side pagination** ile çalışmalı (arama ve filtre desteğiyle)
- Veritabanı şeması **EF Core Migration** ile oluşturulmalı
- Frontend **tek sayfa** (single page) olarak geliştirilmeli
- Frontend'de özet bilgi kartları, sayfalanmış tablo ve ekleme/düzenleme/silme modalları bulunmalı

Backend API

Gerekli endpoint'ler tasarlanmalı ve geliştirilmelidir. Endpoint yapısı, route isimlendirmesi, DTO yapıları ve parametre seçimleri tamamen geliştiriciye bırakılmıştır.

Frontend Arayüzü

Tek bir React sayfası geliştirilecektir. Sayfa ismi, bileşen yapısı ve görsel tasarım size aittir. Aşağıdaki unsurlar bulunmalıdır:

- **Özet kartları** (içeriği sizin tasarımınıza bağlı)
- **Sayfalanmış tablo** — server-side pagination (varsayılan 25 kayıt), arama ve filtreleme desteği
- **Ekleme / düzenleme modalı** (MUI Dialog)
- **Silme onay modalı**

Uyulması Zorunlu Kurallar

1. HTTP Metod Kuralları

Production sunucularında **PUT ve DELETE** HTTP metodları desteklenmemektedir.

```
// DOGRU
[HttpGet("by-company/{companyId}")]
public async Task<IActionResult> GetByCompany(string companyId) { ... }

[HttpPost("create")]
public async Task<IActionResult> Create([FromBody] CreateDto dto) { ... }

[HttpPost("update")]
public async Task<IActionResult> Update([FromBody] UpdateDto dto) { ... }
```

```
[HttpPost("delete")]
public async Task<IActionResult> Delete([FromBody] DeleteDto dto) { ... }
```

```
// YANLIS – Kullanılmayacak
[HttpPut("{id}")] // YASAK
[HttpDelete("{id}")] // YASAK
```

2. Multi-Tenant Security

Her endpoint'te **CompanyId** bazlı filtreleme ve yetkilendirme **zorunludur**. Bir şirketin verileri başka bir şirket tarafından erişilebilir olmamalıdır.

```
// GET endpoint'lerinde CompanyId kontrolu
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id, [FromQuery] string companyId)
{
    var entity = await _manager.GetByIdAsync(id, companyId);
    if (entity == null || entity.CompanyId != companyId)
        return Forbid();
    return Ok(entity);
}

// POST endpoint'lerinde CompanyId dogrulama
[HttpPost("update")]
public async Task<IActionResult> Update([FromBody] UpdateDto dto)
{
    var existing = await _manager.GetByIdAsync(dto.Id);
    if (existing == null) return NotFound();
    if (existing.CompanyId != dto.CompanyId) return Forbid();
    // ...
}
```

- CompanyId eksikse: BadRequest (400)
- CompanyId uyuşmuyorsa: Forbid (403)

3. Entity Framework Core Zorunlu — Raw SQL Yasak

Tüm veritabanı işlemleri **Entity Framework Core** ile yapılmalıdır. Raw SQL, Dapper, ADO.NET, SqlCommand, SqlConnection **kesinlikle kullanılamaz**.

```
// DOGRU
var items = await _context.MyEntities
    .Where(x => x.CompanyId == companyId && !x.IsDeleted)
    .OrderByDescending(x => x.CreatedAt)
    .ToListAsync();
```

```
// YANLIS
var cmd = new SqlCommand("SELECT * FROM MyTable", connection); // YASAK
var data = connection.Query<T>("SELECT ..."); // Dapper YASAK
```

4. EntityState.Modified Zorunlu

Güncelleme işlemlerinde `EntityState.Modified` **mutlaka** kullanılmalıdır. Aksi halde production ortamında değişiklikler kaydedilmez.

```
// DOGRU
entity.Name = dto.Name;
entity.UpdatedAt = DateTime.UtcNow;
_context.Entry(entity).State = EntityState.Modified;
await _context.SaveChangesAsync();

// YANLIS – Production'da calismaz
entity.Name = dto.Name;
await _context.SaveChangesAsync(); // Degisiklik kaydedilmez!
```

5. Server-Side Pagination Zorunlu

Listeleme endpoint'lerinde tüm veriyi frontend'e gönderip client-side sayfalama yapmak **yasaktır**.

```
// DOGRU – Server-side pagination
var totalCount = await query.CountAsync();
var data = await query
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();

return Ok(new {
    success = true,
    data,
    totalCount,
    page,
    pageSize,
    totalPages = (int)Math.Ceiling((double)totalCount / pageSize)
});
```

6. Naming Convention

Katman	Kural	Örnek
C# (Backend)	PascalCase	CompanyId, ProductName, GetByCompany

Katman	Kural	Örnek
TypeScript (Frontend)	camelCase	companyId, productName, getByCompany
Request Body (Frontend → Backend)	PascalCase	{ CompanyId: "abc", ProductName: "Ürün-1" }
Response (Backend → Frontend)	PascalCase döner, frontend dönüştürür	response.data.CompanyId → companyId

İş Bitiminde

Geliştirme tamamlandığında, ayrı bir **CALISMA_RAPORU.md** dosyası oluşturulmalıdır. Bu dosyada:

- Ne yapıldığının kısa özeti
- Kullanılan teknolojiler ve versiyonları
- Karşılaşılan sorunlar ve çözüm yolları
- Mimari kararlar ve nedenleri
- Yapay zeka kullanıldıysa hangi aşamalarda kullanıldığı

açıkça belirtilmelidir.

Değerlendirme Kriterleri

Kriter	Ağırlık
Kurallara uyum (HTTP, Multi-Tenant, EF Core, Pagination)	%25
Kod kalitesi ve mimari uyum (katmanlı yapı, temiz kod)	%20
Özgün tasarım ve problem çözme yaklaşımı	%20
Frontend UI/UX (kullanılabilirlik, görsel kalite)	%15
Server-side pagination doğru çalışıyor mu	%10
CALISMA_RAPORU.md detay ve kalitesi	%10

Bu döküman dışında herhangi bir soru veya belirsizlik durumunda iletişime geçilmelidir.